

CryptoTrace

Technical Documentation and Demo Guide

Read-only documentation of the checked-in **main** branch

Generated 1 September 2026

This document describes only behavior verified from source code, repository fixtures, and terminal checks. It does not claim functionality that is absent from this branch.

Contents

Section	Topic
1	Project overview
2	Architecture and data flow
3	Frontend
4	Backend and API
5	Blockchain and tracing
6	Risk and VASP
7	Persistence and reports
8	Authentication
9	Testing and demo procedure
10	Security and limitations
11	Roles and technology stack
12	Glossary
13	Two-minute explanation and judge Q&A;
14	Current status

1. Project overview

CryptoTrace is a prototype for investigating cryptocurrency movement. An investigator supplies a wallet address and optional case information. The system retrieves public transaction data, builds a directed wallet graph, applies FIFO-style attribution, calculates explainable risk indicators, checks destination addresses against a local VASP label file, stores the result, and exposes report links.

Audience: investigators, analysts, students, and demo judges. **Output:** a JSON investigation response containing evidence, graph data, risk information, VASP matches, and PDF/CSV links.

A blockchain wallet is an address, not a person. The project therefore produces investigative leads only; it does not perform KYC, identify real-world people, freeze funds, or prove criminality.

What is actually implemented on main

Area	Verified statement
Backend	FastAPI application in backend/main.py with auth, cases, trace, reports, and health routes.
Blockchain	Ethereum/Etherscan adapter and cached Ethereum fixture. No TRON adapter is present on main.
Tracing	NetworkX graph construction and bounded BFS expansion in graph_utils.py. trace_engine.py exists but is not called by the current /trace route.
Frontend	React + TypeScript + Vite UI with authentication, trace form, graph, evidence table, case history, and report links.
Storage	SQLite users/cases tables; investigation payloads are stored as JSON.
Reports	ReportLab PDF and Python csv module CSV generation.

2. Architecture and data flow

The checked-in main branch follows this concrete path:

```
Browser
-> Vite proxy (frontend/vite.config.ts)
-> FastAPI (backend/main.py)
-> /trace (backend/api/trace_impl.py)
-> fetcher.py -> Etherscan adapter or eth_cache.json
-> normalize_etherscan_raw()
-> build_graph_from_txs() / bfs_subgraph_from_graph()
-> apply_fifo_attribution()
-> VASP exact lookup + risk calculation
-> sqlite persistence
-> JSON response + report URLs
-> React graph/evidence/result panels
```

Step-by-step connections

Step	File/function	Input -> output
1. Form	frontend/src/components/CaseForm.tsx: handleTrace	Wallet and optional fields -> POST /trace JSON.
2. Route	backend/api/trace_impl.py: trace	TraceRequest -> validated wallet list and response assembly.
3. Fetch	backend/services/fetcher.py; backend/adapters/etherscan_adapter.py	Address -> Etherscan-shaped transactions or cache rows.
4. Normalize	backend/services/normalizer.py: normalize_etherscan_raw	Raw fields/value in wei -> tx_hash, from, to, amount, asset, timestamp, block.
5. Graph	backend/services/graph_utils.py	Normalized transactions -> NetworkX graph, bounded subgraph, node/edge payload.
6. Evidence	backend/services/attribution.py: apply_fifo_attribution	Transactions and suspect wallets -> traceable/unclassified evidence rows.
7. Intelligence	vasp_matcher.py and fraud_detector.py	Evidence -> exact VASP labels and risk profile.
8. Store	persistence.py -> sqlite_store.py	Complete response -> cases.payload JSON.
9. Reports	api/reports.py -> report_generator.py	Stored evidence/summary -> PDF or CSV file response.
10. UI	App.tsx, GraphView.tsx, EvidenceTable.tsx	Response -> summary cards, graph, evidence, links.

3. Frontend

The frontend is React 18 written in TypeScript and bundled by Vite. Styling is a single CSS file at frontend/src/styles.css. There is no frontend test runner in package.json; the available automated validation is the Vite production build.

Entry point and state

frontend/src/main.tsx mounts App. App.tsx owns authentication state, the current investigation response, case history, loading state, and authentication errors. Authentication is kept in localStorage under the key cryptotrace-auth. When a token exists, App requests /cases and passes the token as a Bearer header.

Investigation form

CaseForm sends case_id, case_name, chain, wallets, source_wallet, optional tx_hash, optional amount, optional currency, and max_hops. On this branch the visible chain selector is ETH, BSC, POLYGON, and BASE, but the backend route validates every wallet as an Ethereum address and normalizes through the Ethereum path. TRON is not implemented on main.

Result display

? Summary values are recomputed from evidence rows in App.tsx rather than consistently reading backend summary totals.

? GraphView expects graph node objects with id/type/cluster fields and renders an SVG relationship view.

? EvidenceTable accepts rows and displays abbreviated addresses, transaction links, amount, asset, timestamp, historical value, and VASP.

? Case history loads /cases and selecting an item requests /cases/{case_id}.

? PDF and CSV links are taken from the response report_url and csv_report_url fields.

Confirmed main-branch UI limitations include hardcoded ETH in the summary cards, visible placeholder text such as ?Loading?? and ??????????, prefilled demo credentials in App.tsx, and locally generated JSON/CSV download buttons in addition to the backend report endpoints.

4. Backend and API

Application entry point

backend/main.py creates the FastAPI app, initializes seed users, configures CORS for localhost/127.0.0.1 port 5173, installs request logging middleware, and includes routers for authentication, cases, schema, reports, trace, and health.

Endpoints

Method	Endpoint	Purpose	Important status behavior
GET	/health	Liveness check.	200 {"status":"ok"}.
POST	/auth/register	Create user, hash password, issue JWT.	200; 400 missing data; 409 duplicate email.
POST	/auth/login	Verify credentials and issue JWT.	200; 401 invalid credentials.
GET	/auth/me	Decode Bearer JWT and return user.	200; 401 invalid token; 404 user missing.
POST	/trace	Fetch, normalize, graph, attribute, score, label, persist.	200 success; 400 invalid/empty wallet list; 404 no data; 500 unexpected exception.
GET	/cases	List case metadata, optionally scoped to authenticated user.	200.
GET	/cases/{case_id}	Return stored payload.	200; 403 ownership failure; 404 missing case.
GET	/reports/{case_id}.pdf	Generate and return PDF from stored case.	200; 404 missing case; 500 generation error.
GET	/reports/{case_id}.csv	Generate and return CSV from stored evidence.	200; 404 missing case; 500 generation error.

/trace request and response

TraceRequest is defined in backend/api/trace_impl.py. case_id is optional on this branch; chain defaults to ETH; wallets is a list; source_wallet, target_wallet, address, tx_hash, amount, currency, start_date, and end_date are optional; max_hops defaults to 3 and is capped at 5 by Pydantic.

The response includes case_id, status, wallets, source/target fields, chain, seed_tx, requested fields, summary, risk_profile, graph, graph_metrics, graph_hash, VASP matches, evidence, destination_wallets, wallet_cluster, wallet_clusters, node_roles, report URLs, data_source, fallback_used, evidence_checksum, legal_notice, and audit_log.

5. Blockchain and tracing

Ethereum data path

backend/adapters/etherscan_adapter.py calls https://api.etherscan.io/v2/api with module=account, action=txlist, chainid, address, sort=asc, and an API key read from configuration. Values are converted from wei to ETH. fetcher.py uses the cache when use_cache is true, which is the default demo behavior because backend/config.py defaults DEMO_MODE=true.

The repository contains backend/data/eth_cache.json with three public Ethereum transaction records. The records include transaction hashes, source/destination addresses, ETH amounts, timestamps, block numbers, and explorer URLs. They are fixture data and must not be described as a live result.

Live versus demo

Mode	Configuration	Actual behavior
Demo/cache	DEMO_MODE=true (default) or fetch_transactions(use_cache=True).	Reads eth_cache.json. The current cache reader returns the fixture list without filtering by address.
Live	USE_ETHERSCAN=true, DEMO_MODE=false, ETHERSCAN_API_KEY configured.	Calls Etherscan V2 through retry wrapper. Empty/failing live results are surfaced by /trace as a 404.
TRON	No TRON adapter, settings field, or TRON normalizer exists on main.	Not implemented / not verified on this branch.

Tracing behavior on main

The current /trace route calls `build_graph_from_txs()` and `bfs_subgraph_from_graph()` from `graph_utils.py`. BFS collects every successor edge reachable within `max_hops`. It does not call `bounded_trace()` from `backend/graph/trace_engine.py`, whose implementation follows largest outbound edges. Therefore largest-outgoing selection and an authoritative ordered path are not verified for the current main branch.

The graph payload is built from the bounded BFS nodes/edges and evidence rows. FIFO attribution annotates every normalized transaction, then the current route returns the evidence list without applying a separate traced-edge filter. This is a confirmed integration limitation: evidence may not be limited to only the selected path.

6. Risk and VASP

Risk

`backend/services/fraud_detector.py` calculates a multilayer probability from evidence and wallets. It considers direct exposure, VASP hits, unclassified flow, propagation/unique-address count, and wallet binding. The result is rounded and capped at 99. It also returns confidence, risk factors, and a candidate wallet. A risk probability is an investigative heuristic, not proof of fraud or identity.

VASP

`backend/services/vasp_matcher.py` loads the first readable file from `backend/data/vasp_labels.json` or `backend/vasp/vasp_labels.json`. It normalizes list-shaped labels to an address-keyed dictionary and performs a case-insensitive exact address lookup. A match may carry entity, confidence, source, and `source_date`. No match becomes UNKNOWN. The data is local/static; there is no live VASP API.

The current main route calls `match_vasp_for_address()` without a chain parameter. Chain-aware matching is therefore not verified on this branch. Attribution must remain UNKNOWN when no exact local label exists; inventing an organization would be unsafe.

7. Persistence and reports

SQLite

`backend/services/sqlite_store.py` creates users and cases tables in `backend/data/cryptotrace.db`. Users contain email, password_hash, full_name, and created_at. Cases contain a unique case_id, optional user_id, JSON payload, created_at, and updated_at. `save_case` uses `INSERT ... ON CONFLICT(case_id) DO UPDATE`, so repeating a case ID updates rather than duplicates the row.

Trace persistence is best-effort: the /trace route catches persistence exceptions and silently continues. This means a successful-looking response is possible even if storage fails; persistence should be verified separately through /cases or /cases/{case_id}.

Reports

backend/api/reports.py loads the stored case and calls the existing ReportLab/Python CSV generator in backend/services/report_generator.py. No second generator exists. The current generator includes evidence, VASP information, risk flags, checksum/legal notice, and a summary. However, it hardcodes ETH in the PDF summary and the CSV defaults chain to ETH when evidence/summary do not supply a chain. A bounded trace path is not explicitly rendered by the current report generator.

Reports are written to backend/reports/report_{case_id}.pdf/csv and returned with FileResponse. The frontend also has local JSON and CSV download helpers; these are separate from the authoritative backend report endpoints and should not be confused with them.

8. Authentication

Registration and login are implemented in backend/api/auth.py. Passwords are processed with PBKDF2-HMAC-SHA256 using a configurable PASSWORD_SALT. JWTs are signed with HS256 using JWT_SECRET and expire after JWT_TTL_MINUTES. The default values in auth.py are development fallbacks and must be replaced through environment configuration for real deployment.

The frontend sends POST /auth/login or /auth/register through the Vite proxy and stores the returned token. /cases, /cases/{case_id}, and trace requests may include the Bearer token. Cases are user-scoped when a valid user is resolved.

Authentication was reproduced against a clean backend: seeded demo and named accounts returned HTTP 200; an incorrect password returned HTTP 401; login through the Vite proxy returned HTTP 200 when the backend was running. The observed browser ?Authentication failed? state was associated with /cases returning HTTP 500 while the backend was unavailable, not with a login failure.

The frontend currently pre-fills a demo email and password. This is a confirmed security/UI issue, not a reason to bypass authentication.

9. Testing and demo procedure

Commands verified

```
python -m pytest backend/tests -q
# main branch result: 11 passed

cd frontend
npm run build
# result: Vite production build succeeded
```

The build emitted a Vite CJS API deprecation warning. No frontend automated test script is declared. Docker was not used in this audit.

Safe browser demo

- ? Start backend from backend/: `python -m uvicorn main:app --reload --port 8000`.
- ? Check `http://127.0.0.1:8000/health` and expect `{"status":"ok"}`.
- ? Start frontend from frontend/: `npm run dev -- --host 127.0.0.1 --port 5173`.
- ? Open `http://127.0.0.1:5173` and log in with a configured/seeded account without placing credentials in documentation.
- ? Use the existing ETH fixture address from DEMO_VERIFICATION.md only if the cache and route behavior are the intended demo target.
- ? Inspect the response, graph, evidence, risk, VASP, case history, PDF, and CSV links.
- ? Use an invalid address to verify HTTP 400 and a no-history address to verify the configured 404 path.

What cannot be honestly demonstrated on main

- ? TRON tracing: no adapter or TRON normalization path exists.
- ? Largest-outgoing path selection: /trace uses BFS, not `trace_engine.bounded_trace()`.

? Guaranteed bounded evidence: the route does not filter evidence to selected graph edges.

? Cross-chain bridge tracing: no bridge adapter or cross-chain path model exists.

? A verified VASP-positive case: local labels exist, but a positive end-to-end match was not established by the current main-branch checks.

10. Security and limitations

Concern	Implemented protection	Remaining limitation
Secrets	Environment variables are supported for API/JWT/password settings; this document exposes none.	Development fallback secrets exist in auth.py/config.py.
Input validation	Pydantic request models and Ethereum address validation reject malformed wallets.	Validation is Ethereum-only on main.
Unknown data	No-data path returns an error rather than fabricated transactions when the active route has no rows.	Demo cache is not address-filtered by fetcher.py on main.
Evidence scope	BFS is hop-bounded.	Evidence is not explicitly reduced to the selected path edges.
Auth	JWT Bearer tokens and password hashing.	Frontend prefilled credentials; error handling for stale tokens is weak.
Reports	Reports are generated from stored case evidence.	ETH defaults and missing explicit trace path reduce cross-chain/report fidelity.
External calls	Timeouts and retry wrapper exist for Etherscan.	Some broad exception handling converts failures to empty results.

Bridge status: NOT IMPLEMENTED / NOT VERIFIED. Live data status: ETH requires a valid configured key; no key value was printed or recorded here. Backup fixture status: no independent second fixture is established in the checked-in main branch. Docker status: not verified in this environment.

11. Roles and technology stack

The checked-in main branch does not contain the shared role-assignment document referenced by later project notes. The following task sequence is therefore a repository-context summary, not a claim that role metadata is present in source:

Milestone	Responsibility summary	Observed integration
M1	Database/storage	SQLite users/cases persistence.
M2	Blockchain integration	ETH adapter/fetcher/cache on main; TRON absent.
M3	Tracing/integration	NetworkX graph and attribution services; current route uses BFS utility.
M4	VASP	Local exact-match label lookup.
M5	Risk/fraud	Multilayer heuristic probability and factors.
M6	Frontend	React form, result UI, graph, evidence, auth, history, reports.
Demo/docs/QA	Documentation and validation	README, DEMO_VERIFICATION.md, docs notes.

Technology stack

Layer	Technology	Where used / why
Language	Python	FastAPI backend and services.
API	FastAPI + Pydantic	HTTP routes and request validation.
Frontend	React 18 + TypeScript	Stateful investigation UI.
Build	Vite	Frontend development server and production build.
Graph	NetworkX	Directed transaction graph and bounded BFS.
Storage	SQLite	Local users and case payload persistence.
Blockchain	Etherscan V2 API + JSON fixture	Public ETH history and reproducible demo data.
Reports	ReportLab + csv	Backend PDF/CSV exports.
Deployment	Docker Compose	Configuration exists, but Docker was not verified in this audit.

12. Beginner glossary

Term	Simple explanation in CryptoTrace
API	A program interface used by software to exchange data. CryptoTrace exposes FastAPI endpoints such as /trace.
Adapter	Code translating one provider format into the project format. etherscan_adapter.py maps Etherscan fields.
Blockchain	A public ledger of transactions.
Wallet	A blockchain address that sends or receives funds.
Transaction hash	A unique identifier for a transaction.
Block	A numbered group of blockchain transactions.
Normalization	Converting provider-specific fields into one internal transaction shape.
Hop	One wallet-to-wallet step in a traced flow.
Evidence	A transaction row with attribution fields such as traceable_amount.
VASP	Virtual Asset Service Provider, such as an exchange or custodian.
Risk probability	A heuristic percentage indicating suspicious-flow signals, not a criminal finding.
Fixture/cache	Stored data used for repeatable local testing.
Persistence	Saving a result so it can be retrieved later.
Graph/node/edge	A relationship model: wallets are nodes and transfers are directed edges.
JWT	A signed token used to carry authenticated session claims.
CORS	Browser policy allowing configured frontend origins to call the backend.
PDF/CSV	Human-readable report and tabular export formats.

13. If I had to explain CryptoTrace in 2 minutes

CryptoTrace is a public-blockchain investigation prototype. An analyst enters a wallet and case details in the React interface. The frontend sends JSON to a FastAPI backend. The backend reads Ethereum transactions from Etherscan or the local fixture, normalizes them, builds a directed graph, expands it within a hop limit, and applies conservative FIFO attribution to estimate which outgoing value is connected to the investigated wallet. It then checks destination addresses against a local VASP label file and calculates explainable risk factors. The complete response is stored in SQLite and can be retrieved through case history. The same stored evidence is used to generate PDF and CSV reports, while the frontend renders the graph, evidence table, risk result, VASP outcome, and report links.

The safe qualification is important: the current main branch is Ethereum-focused. TRON, arbitrary cross-chain bridge tracing, verified largest-outgoing selection, and guaranteed path-only evidence are not implemented or verified here. VASP UNKNOWN means no exact local label matched; it does not mean the address is safe.

Questions I should be able to answer

Question	Accurate short answer
What does the user submit?	A case ID, Ethereum wallet list, optional source/target, transaction hash, amount, currency, and max hops.
Where does data come from?	Etherscan V2 in configured live mode or eth_cache.json in demo/cache mode.
Why normalize?	To give graph, attribution, risk, and reports one stable transaction shape.
What is a hop?	One directed transfer from one wallet node to another.
Is tracing largest-outgoing on main?	No. trace_engine has such logic, but the current /trace route uses BFS graph_utils.
Is evidence path-bounded on main?	Not guaranteed; the route returns FIFO evidence without selected-edge filtering.
What does UNKNOWN VASP mean?	No exact address match was found in the local labels.
Does risk prove fraud?	No. It is an explainable investigative heuristic.
How are passwords stored?	PBKDF2-HMAC-SHA256 hashes with a configurable salt.
How is login represented?	A signed HS256 JWT returned by /auth/login.
Where are cases stored?	SQLite cases.payload as JSON, keyed by unique case_id.
Can repeated case IDs duplicate rows?	No; SQLite upsert updates the existing case.
How are reports made?	The backend calls the existing ReportLab/Python CSV generator.
Does the frontend call reports directly?	It opens the URLs returned by /trace.
Does TRON work on main?	Not implemented / not verified.
Does bridge tracing work?	Not implemented / not verified.
What happens for invalid wallets?	Ethereum address validation returns HTTP 400.
What happens for no transaction data?	The active route returns HTTP 404 with an explanatory detail.
What should never be claimed?	Private KYC access, real-person identity, definitive criminality, or unsupported chain/bridge behavior.
What is the first safe operational check?	Start the backend and call /health before opening the frontend.

14. Current status

Component	Status	Evidence	Limitation
Frontend	PARTIAL	React/Vite build passed; App, CaseForm, GraphView, EvidenceTable exist.	ETH-focused UI; placeholders, prefilled credentials, hardcoded ETH.
Backend/API	PASS	FastAPI routes and /health/auth checks succeeded.	Persistence errors are swallowed in /trace.
ETH	PASS	Etherscan adapter, normalizer, fixture, and tests.	Live mode needs a valid key.
TRON	FAIL	No adapter/normalizer/configuration on main.	Must not be claimed.
Trace engine	PARTIAL	trace_engine.py exists; graph_utils BFS is used by /trace.	Largest-outgoing path not connected.
Multi-hop	PARTIAL	BFS max_hops expansion exists.	Ordered path and selected-edge evidence are not authoritative.
Risk	PASS	fraud_detector.py returns probability/factors/candidate.	Heuristic only.
VASP	PASS	Local exact-match loader/matcher.	Static coverage; no chain-aware match on main.
Database	PASS	SQLite schema, upsert, case retrieval.	Historical/demo pollution needs operational review.
Authentication	PASS with concerns	Direct and proxy login returned 200; bad password 401.	Development defaults and prefilled UI credentials.
Graph	PARTIAL	NetworkX graph payload and SVG renderer.	Frontend/backend shape alignment is not fully proven on main.
Evidence	PARTIAL	FIFO evidence table and attribution.	Current route does not enforce selected-path filtering.
PDF	PARTIAL	ReportLab endpoint generates files.	ETH defaults; no explicit trace-path section.
CSV	PARTIAL	CSV endpoint generates evidence export.	Chain defaults can be ETH.
Demo mode	PASS	DEMO_MODE defaults true and eth_cache.json exists.	Cache reader is not address-aware.
Live mode	PARTIAL	Etherscan V2 and retry wrapper exist.	Requires configuration and was not live-verified here.
Testing	PASS for existing suite	11 backend tests passed; frontend build passed.	No frontend automated tests.
Documentation	PARTIAL	README and DEMO_VERIFICATION.md exist.	Docs do not fully match main implementation.

Final conclusion: the repository is a working ETH-oriented prototype with authentication, case storage, graph/risk/VASP/report components and a passing existing test suite. It is not a fully integrated multi-chain tracing system on the checked-in main branch. The most important next engineering task is to reconcile the intended feature branch with main before making isolated fixes, then reconnect the authoritative tracing engine and verify evidence/report contracts end to end.